

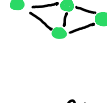
Graph definitions → a graph can be defined using a pair (V, E) where V is the set of vertices and E is the set of edges

if we take $E = \{(u, v) \in V^2 \mid (u, v) \in E \wedge (v, u) \in E\}$ we have an undirected graph otherwise we have a directed graph

We define a **simple graph** as an undirected graph with no self loops

Two nodes are called **adjacent** if they have exactly one common endpoint

for example →  A and C are adjacent

Order and Size of a graph → the order is the number of vertices $|V|$ and the size is the number of edges $|E|$, for example  has order = 4 and size = 5.

Density of a graph → it is the ratio between the graph's size and the number of all possible nodes, we use two different formulas for directed and undir.

Directed → $\text{dens}(G) = \frac{2l}{n(n-1)}$ **Undirected** → $\text{dens}(G) = \frac{2l}{n(n-1)}$

The density is particularly important to identify the type of graph we are working with. A **Sparse graph** has size $(l) : O(n)$, a **Dense graph** has $O(n^2)$.

we use different data structures depending on the type.

Reciprocity of a graph → this is only applicable to digraphs and it measures the probability of seeing (u, v) given $(v, u) \rightarrow \text{rec}(G) = \frac{l \leftrightarrow}{l}$ where $l \leftrightarrow$ is the number of reciprocal links.

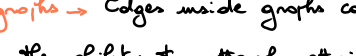
Operations on graphs → Since we defined graph with sets, we can also use set operations on them.

Complement → $G^* = \{(u, v) \in V(G)^2 \mid (u, v) \notin E(G)\}$

Union → $G \cup G' = E(G) \cup E(G')$ **Intersection** → $G \cap G' = E(G) \cap E(G')$

Difference → $G \setminus G' = E(G) - E(G')$

Line graph → the line graph of G is another graph where the edges of G become the nodes of $L(G)$ and the new nodes are connected if they are adjacent in the first graph.



Edge attributed graphs → Edges inside graphs can be used for modelling relations so we also need the ability to attach attributes to them. In order to do that we define a new set of functions called **attribute functions** $\{f_1 \dots f_d\}$ such that $f_i: E \rightarrow S_i$ where S_i is a generic set of values. These are the building blocks for attaching weights to edges. $[w_1 \dots w_d]$ If we use each feature vector on the i -th row of a matrix, we create the **edge feature matrix** $E \in \mathbb{R}^{l \times d}$

Node attributed graph → what we just said for edges also applies to nodes, the resulting matrix is called **node feature matrix** $U \in \mathbb{R}^{n \times d_v}$

Representing graphs → there are multiple viable ways to represent a graph and they mostly depend on the use case.

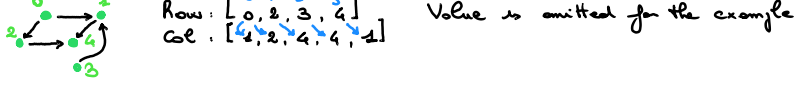
Adjacency matrix → $A \in \mathbb{R}^{n \times n}$ where $A_{u,v} = 1$ if $(u, v) \in E(G)$, it is always symmetric for undirected graphs ($A = A^T$). Very good for dense graphs, not so much for sparse graphs.

Coordinate list (COO) → Only stores the set of entries

A	B	C	E
B	D	D	C
1	1	1	1

Compressed sparse row (CSR) → it is a little trickier than the previous two, it has a 3-index structure that is organized as follows: ① Row $\in \mathbb{R}^n \rightarrow$ index for node at j indicating where the set of nodes that are incident to j starts.

② Column $\in \mathbb{R}^l \rightarrow$ ordered set of links. ③ Value $\in \mathbb{R}^l \rightarrow$ attribute fn / weight for each link



Adjacency list → the entire idea is to use a list or set to store the nodes linked to a node u , using the graph above we have:

$0 \rightarrow \{1, 2\}$; $1 \rightarrow \{4\}$; $2 \rightarrow \{4\}$; $3 \rightarrow \{1\}$; $4 \rightarrow \{1\}$

Edge list → Simple list of tuples → $[(0, 1, \text{value?}), (0, 2), (1, 4) \dots]$

Incident matrix → rectangular matrix $\mathbb{R}^{n \times l}$ where the values are as follows:

Undirected → $\{u, v\} \in E(G) \Leftrightarrow A_I(u, (u, v)) = 1 \wedge A_I(v, (u, v)) = 1$, else 0

Directed → $(u, v) \in E(G) \Leftrightarrow A_I(u, (u, v)) = 1 \wedge A_I(v, (u, v)) = -1$, else 0

Example with the graph above (directed)

	e_1	e_2	e_3	e_4	e_5	1	-1
						2	0
						3	0
						4	0

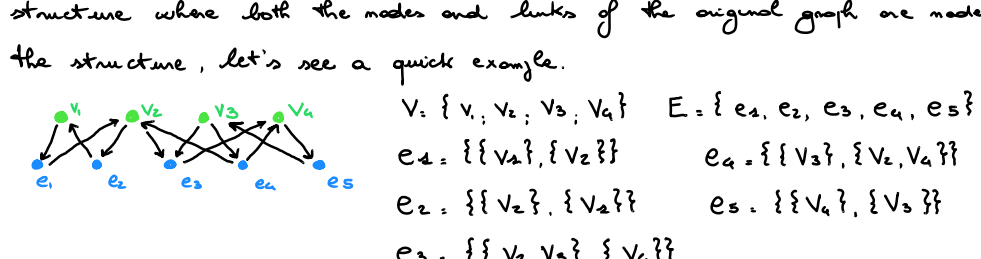
Hypergraphs → So far we have seen graphs where edges could only connect two nodes, if we remove this rule we have $e = \{v_1 \dots v_k\}$ with $k \geq 2$, also called a hyperedge. These new components allow us to model more complex relationships.

Directed Hypergraph → $H = (V, E)$; $E = 2^V \times 2^V$ (powerset of $V \rightarrow$ set of all possible subsets of V). $e \in E$ has the form (U, W) where U is called **tail** and W is called **head**.

Undirected Hypergraph → Slightly simpler because $e \in E$ has form $\{U, W\} = U \cup W$ therefore $E \subseteq 2^V$

Order, Size and Density for hypergraphs → order is still $n = |V|$ and size is still $l = |E|$; the density is calculated as follows: $\text{dens}(G) = \frac{l}{2^n(2^n-1)}$ (directed) ; $\text{dens}(G) = \frac{l}{2^n}$ (undirected)

Representing a hypergraph with an incidence structure → the idea is to create a structure where both the nodes and links of the original graph are nodes of the structure, let's see a quick example.



this can be easily translated into a matrix similar to the one seen for classical graphs.

	e_1	e_2	e_3	e_4	e_5	
V_1	1,0	0,1	0,0	0,0	0,0	1° component = 1 if tail
V_2	0,1	1,0	1,0	0,1	0,0	2° component = 1 if head
V_3	0,0	0,0	1,0	1,0	0,1	
V_4	0,0	0,0	0,1	0,1	1,0	

the membership of vertices in sets as in hypergraphs can be represented by using a **Bijunctate graph** or **Two-mode network**.

Bijunctate graph → a network where we have two types of nodes and edges can only connect nodes of **different types**; obviously a node can only have one type.

the incidence matrix for a bijunctate graph is $B_I \in \mathbb{R}^{|V_1| \times |E|}$ and the values are

$B_{u,c} = \begin{cases} 1 & \text{if } u \in c \\ 0 & \text{otherwise} \end{cases}$		<table border="1" data-bbox="497 3804 652 3912"> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td></tr> </table>	1	0	0	1	1	0	0	1	0	0	0	1
1	0	0												
1	1	0												
0	1	0												
0	0	1												

Bijunctate structures are not always great for ML; we'd like to map such structures on "normal" graphs. We can do that with approaches such as **one mode projection**

Interacting networks → most real systems are way more complex than normal graphs, therefore we need structures to represent such systems.

General multi-layer network → defined by a triple $M = (Y, \vec{G}, \vec{B})$ where:

1 → $Y = \{\alpha \mid \alpha \in \{1 \dots M\}\}$ is the set of layers.

2 → $\vec{G} = \{G_1 \dots G_M\}$; $G_\alpha = (V_\alpha, E_\alpha)$ is the set of graphs where the links inside a graph are called **intralinks**.

3 → $\vec{B} \in M \times M$ is the set of bijunctate graphs with $B_{\alpha\beta} = \{V_\alpha, V_\beta, E_{\alpha\beta}\}$

Multilayer network → multi layer structure that maps 1 to 1 nodes between layers (replica nodes), interlinks can only connect replica nodes.

A multilayer network can be both **with** and **without** interlinks. $V \Rightarrow V_d = \{(1, 2)\}$

With → used when replica nodes have a distinct identity $\rightarrow \{?\}$

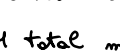
Without → used when there is no distinction between replicas $\rightarrow M = (Y, \vec{G}, \vec{B})$

For multilayer networks **with** interlinks, we can represent them using a single **Synapse-adjacency matrix** of the following form.

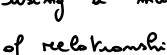
$A \in \mathbb{R}^{n \times M \times n \times M}$ where $n = |V|$ and M is the number of layers. The matrix is made of blocks that define the interlinks and intralinks.

$A = \begin{bmatrix} A^1 & I & I \\ \vdots & \vdots & \vdots \\ I & I & A^M \end{bmatrix}$ the identity matrices $I \in \mathbb{R}^{n \times n}$ define the interlinks between the layers; the $A^1 \dots A^M$ matrices are instead used for intralinks

$A_{i\alpha, j\beta} = \begin{cases} A^i_{\alpha\beta} & \text{if } \alpha = \beta \\ \delta(i, j) & \text{if } \alpha \neq \beta \end{cases}$

For multilayer networks **without** interlinks, we can encode them using one adjacency matrix for each layer (M total matrices). We can also represent them using a multi (di)graph →  The color is used to distinguish the type of relationship.

Multi slice networks → a more specific type where the interlinks can only exist in a temporal succession. $E_{d, d+1} = \{(i, d), (i, d+1) \mid i \in \{1 \dots n\}\}$

 the synapse adjacency matrix is very similar to the one above

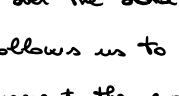
 but the identity matrices are only on the blocks I above the diagonal.

Network of networks → a multilayer network with a **supernetwork**, basically the supernetwork decides which layers have their replica nodes linked. It is used to model real world countours over the data.

Time in networks → in our case, time allows us to model real world processes that are dynamic in nature. We can represent the evolution over time with a tabular form, a small example is:

Using the tabular form (from, to, start, duration) we define the table below

a	b	1	1
b	c	2	3
c	a	3	4

 which, in graph form, is →  also called **interval graph**

the example above is also called **link stream** and assumes that the nodes are active through the entire lifetime of the graph.

We can also add a table for node lifetimes if needed

For more specific use cases, where the data we have comes from snapshots, we can use **time varying graphs**, to which we can apply either coarse or variable resolution to capture the graph's evolution over time.